

Practica calificada 3

1. Contexto

Nota: No pude ejecutar el entorno local durante esta evaluación porque no tenía Docker instalado. Para sustituir esa evidencia, revisé los documentos del repositorio, los contratos de cada servicio y el código fuente. Si tuviera el entorno disponible, seguiría la guía docs/setup-and-usage.md para validar todo en ejecución.

El sistema Tutorías Universitarias permite a un estudiante reservar una sesión con un tutor. Los servicios que participan son: ms-auth (la autenticación), ms-usuarios (los datos de estudiantes y tutores), ms-agenda (disponibilidad y bloqueo de horarios), ms-tutorias (orquesta todo el flujo), ms-notificaciones (envía notificaciones vía RabbitMQ), más PostgreSQL como base de datos y RabbitMQ como broker de mensajería.

El flujo actual va así, el cliente se autentica, solicita una tutoría, mstutorias valida usuarios, bloquea el horario en agenda, registra la tutoría y publica un evento para que ms-notificaciones avise al estudiante.

La nueva funcionalidad agrega mspagos, que se conecta con proveedores externos como Visa o Mastercard. Estos pueden responder lento, rechazar, duplicar respuestas o no responder dentro del tiempo esperado.

2. Problema

El diseño propuesto tiene tres problemas principales:

- Primero, ms-tutorias abre una transacción en su base de datos y sin cerrarla sale a llamar a ms-pagos, que a su vez llama a Visa o Mastercard. Mientras el banco externo no responda, esa transacción sigue abierta ocupando recursos. Si el banco tarda o no responde, todo el sistema queda esperando.
- Segundo, si la conexión con el banco se corta o el cliente cierra la app antes de recibir respuesta, no queda claro si el pago se realizó o no. No hay forma confiable de saberlo.
- Tercero, si el cliente hace clic dos veces o reintenta la solicitud, el flujo completo se ejecuta de nuevo y el estudiante podría ser cobrado dos veces por la misma tutoría.

En resumen, el diseño depende completamente de que un proveedor externo responda rápido y sin errores, lo cual no es realista.

3. Riesgos

Riesgo	Consecuencia
El banco externo no responde a tiempo	La operación queda colgada y bloquea el sistema hasta que se agote el tiempo de espera

El cliente hace doble clic o reintenta	El pago se procesa dos veces y el estudiante es cobrado dos veces
El pago se aprobo pero la tutoría no se guardó	El estudiante pago pero no tiene su tutoría confirmada
La tutoría se guardo pero el pago fue rechazado	La tutoría queda confirmada sin que se haya cobrado
Los datos de la tarjeta pasan por ms-tutorias	Un servicio que no debería manejar información de pago la está viendo y procesando

4. Diseño propuesto

En lugar de que ms-tutorias espere la respuesta del banco dentro de una transacción, el nuevo flujo separa el proceso en dos partes: una síncrona y una asíncrona.

Parte síncrona — ocurre de inmediato:

El cliente solicita la tutoría. ms-tutorias valida que el estudiante y el tutor existan en ms-usuarios, verifica y bloquea el horario en ms-agenda, y luego le pide a ms-pagos que cree una intención de pago. en este punto la tutoría queda en estado pendiente_pago y el cliente recibe una respuesta inmediata sin esperar al banco.

Parte asíncrona — ocurre en segundo plano:

ms-pagos se encarga de comunicarse con visa o mastercard y esperar su respuesta. cuando el banco confirma el pago, ms-pagos publica un evento en rabbitmq. ms-tutorias consume ese evento y cambia el estado de la tutoría a confirmada. luego ms-notificaciones también consume el evento y le avisa al estudiante.

Si el pago es rechazado o falla, ms-tutorias cambia el estado a fallida y libera el bloqueo de agenda.

Estados de la tutoría: de pendiente_pago a confirmada o fallida

Con este diseño, el banco externo nunca controla el flujo interno del sistema. Cada servicio tiene una responsabilidad clara y los datos de pago solo los maneja ms-pagos.

5. Alternativas consideradas

Alternativa 1: Diseño original

Es la opción más simple de entender, pero como se explicó en la sección de problemas, depende completamente de que el banco externo responda rápido. Cualquier demora afecta a todo el sistema. Se descarta porque no es confiable en condiciones reales.

Alternativa 2: Coreografía pura (solo eventos, sin orquestador)

Cada servicio reacciona a eventos sin que nadie coordine el proceso. Es flexible, pero cuando algo falla es muy difícil saber en qué paso quedó la operación y quién es responsable de corregirlo. Se descarta porque el flujo de pagos necesita un orden claro y trazabilidad.

Alternativa 3: Orquestación con pagos asíncronos (propuesta adoptada)

ms-tutorias coordina el flujo síncrono hasta crear la intención de confirmación del banco de forma asíncrona a través de RabbitMQ. Se adopta porque separa bien las responsabilidades, no deja transacciones abiertas esperando al banco y permite rastrear el estado de cada tutoría en todo momento.

6. Contratos
7. Observabilidad
8. Seguridad
9. Despliegue y costos
10. Decisión